

Fundamentos para el Análisis Automático de Lenguaje

Dra. Delia Irazú Hernández Farías

Dr. Luis Villaseñor Pineda

Dr. Manuel Montes y Gómez

Dr. Aurelio López López

Otoño 2026 - Bryan Ezequiel Violante Arriaga

Índice

1. Lenguaje	2
2. Continuación (Lenguaje)	4
3. Recuperación de información (IR)	5
4. Continuación (Vector Space Model)	9
5. Modelos de lenguaje para IR	12

1. Lenguaje

17 de agosto de 2026

Resumen rápido. En esta sesión vimos una intro de qué es el lenguaje, la lingüística, desde sus bases y cómo se ha estudiado a lo largo de la historia, llegando a la lingüística computacional y el procesamiento de lenguaje natural.

Qué es un lenguaje natural?

Un lenguaje natural es un sistema de comunicación que permite a los seres humanos expresar ideas, emociones y conceptos a través de signos y símbolos.

Idea Random

Un sistema de comunicación entre dos animales puede ser considerado lenguaje? O entre seres interespecíficos? Por ejemplo, entre un humano y un perro.

Un lenguaje puede ser influenciado por:

- **Geografía:** Por ejemplo torta es pues una torta en México, pero en Argentina es un pastel.
- **Cultura:** Por ejemplo, en México se dice "aguacate", pero palta en sudamerica
- **Historia**
- **Educación:** "Qué pató mamá kuakuak" es una expresión de barrio
- **Cosas que se descubren**

El procesamiento del lenguaje es cómo el cerebro procesa el lenguaje y cómo entiende el mensaje que se quiere transmitir. Esta habilidad se obtiene a través de la experiencia y la exposición al lenguaje desde una edad temprana y, el cerebro comienza a desarrollar la capacidad de comprender y producir lenguaje a medida que se expone a él.

En ciencias de la computación, el procesamiento del lenguaje natural (PLN) es un campo que se enfoca en desarrollar algoritmos y modelos computacionales que permitan a las máquinas comprender, interpretar y generar lenguaje humano de manera efectiva. El PLN combina técnicas de lingüística, aprendizaje automático e inteligencia artificial para abordar tareas como la traducción automática, el análisis de sentimientos, la generación de texto y la comprensión del lenguaje natural.

Conceptos importantes

- **Lingüística:** Estudio científico del lenguaje, incluyendo su estructura, evolución y uso en la sociedad.
- **Lingüística computacional:** Rama de la lingüística que se enfoca en el desarrollo de modelos y algoritmos para procesar y analizar el lenguaje natural mediante computadoras.
- **Procesamiento de lenguaje natural (PLN):** Campo de estudio que combina técnicas de lingüística, aprendizaje automático e inteligencia artificial para permitir que las máquinas comprendan, interpreten y generen lenguaje humano.

Otras disciplinas que también se relacionan con el estudio del lenguaje son la psicología y las neurociencias.

Para entender el PNL hay varias areas como:

- **Morfología:** Estudio de la estructura de las palabras y cómo se forman a partir de morfemas
- **Léxica:** Estudio del vocabulario de un idioma, incluyendo el significado y uso de las palabras
- **Pragmática:** Estudio del uso del lenguaje en contextos específicos y cómo el significado puede variar según la situación y la intención del hablante.
- **Semántica:** Estudio del significado de las palabras, frases y oraciones, así como las relaciones entre ellos
- **Sintaxis:** Estudio de la estructura de las oraciones y cómo se combinan las palabras para formar oraciones gramaticalmente correctas
- **Fonética:** Estudio de los sonidos del habla, incluyendo su producción, transmisión y percepción
- **Fonología:** Estudio de los sistemas de sonidos en un idioma y cómo se organizan para formar palabras y oraciones

2. Continuación (Lenguaje)

19 de agosto de 2026

Resumen rápido. En esta sesión continuamos con la introducción al curso, como ha evolucionado el NLP que es el NLP multinodal y los modelos de lenguaje, todos estos conceptos así muy por encima, nada super formal, solo para tener una idea de lo que es el curso y a donde vamos a llegar. Igual vimos las áreas o problemas de investigación en PLN.

- **1950:** Alan Turing propone el test de Turing.
- **1957:** Chomsky publica su gramática generativa.
- **1966:** ELIZA, el primer chatbot basado en reglas.
- **1980s:** Auge de los métodos estadísticos sobre corpus.
- **1990s:** Modelos ocultos de Márkov y traducción automática estadística.
- **2013:** word2vec populariza los embeddings de palabras.
- **2017:** Llega la arquitectura Transformer.
- **2018:** BERT y GPT inician los modelos preentrenados a gran escala.
- **2020s:** Modelos de lenguaje grandes de uso masivo.

Al inicio se utilizaban solo textos y reglas para los algoritmos de NLP pero con el avance de la tecnología surgió el **NLP multinodal** (cuando usa solo texto se llama mono nodal), que además de texto, también utiliza imágenes, audio y video para el procesamiento del lenguaje natural.

En nuestra vida diaria el NLP se usa en muchas cosas como:

- Asistentes virtuales (Siri, Alexa, Google Assistant)
- Traducción automática (Google Translate, DeepL)
- Análisis de sentimientos en redes sociales
- Chatbots y atención al cliente automatizada (aunque a veces es deficiente)
- Resumen automático de textos
- Reconocimiento de voz y transcripción

Modelos de Lenguaje

Idea clave

Un modelo de lenguaje es un modelo probabilístico/estadístico que asigna una probabilidad a una secuencia de palabras (o tokens)

Como curiosidad, la fórmula que usualmente se utiliza para predecir la siguiente dado un contexto anterior es

$$P(w_n | w_1, w_2, \dots, w_{n-1}) = \frac{P(w_1, w_2, \dots, w_n)}{P(w_1, w_2, \dots, w_{n-1})} \quad (1)$$

Y pues se parece mucho a la formula de Bayes o a la de probabilidad condicional. Hay dos grandes familias de modelos de lenguaje

- **Basados en n-gramas:** Estiman esa probabilidad contando frecuencias de secuencias de n palabras en un corpus. Simples pero limitados por la escasez de datos (data sparsity) para n grande.
- **Basados en ANNs:** Aprenden una representación continua del contexto y generalizan mejor a secuencias no vistas. Los LLM actuales (GPT, BERT, etc.) son modelos de lenguaje neuronales entrenados sobre enormes corpus.

Areas de investigación en PLN

Hay varias áreas de investigación y/o problemas en PLN:

- Detección de fake news
- Perfilado de autoría
- Detección de discurso de odio (el discurso de odio no siempre es grosero o así, pero es discurso de odio sin ser explícito)
- Detección de humor
- Análisis de sentimiento
- Question answering
- RAG
- Resumen de textos
- Generación de texto
- Text to image
- Image to text (captioning)

3. Recuperación de información (IR)

24 de agosto de 2026

Resumen rápido. En esta sesión comenzamos con un concepto, que al menos para mi es nuevo, el de la recuperacion de informacion. Vimos que es, para que sirve y un par de ejemplos. El contenido de esta seccion de IR es el siguiente

- Definicion de la tarea
- The Vector Space Model
- Modelos de lenguaje para IR
- Evaluacion de IR (Performance)
- Problemas principales y soluciones basicas (Indexing, Query Expansion, Relevance Feedback, Clustering, Diversification)

Introduccion

Para entender de manera intuitiva que es un Sistema de Recuperación de Informacion (IRS), responderemos las preguntas que el Dr planteo en clase...

- **Qué es un sistema de recuperación de información (IR)?** Es un sistema que busca dentro de una colección grande de documentos no estructurados o semiestructurados
- **Que objetivo tiene?** Su objetivo es recuperar documentos relevantes para satisfacer una necesidad de informacion del usuario
- **Que hay dentro de el? Algun sub-proceso?** Pues si, un SRI se descompone tipicamente en sus fases de; Adquisicion, Preprocesamiento del texto, Indexacion, Procesamiento de la query, Recuperacion, Ranking Presentacion de resultados, Retroalimentacion.
- **Como evaluar su desempeno?** Se busca maximizar dos cosas a la vez, la precision (devolver lo que sea relevante), exhaustividad o recall (que no se le escape info relevante que si exista)
- **Porque los resultados no siempre son relevantes?** Porque el matching lexico no siempre es lo mismo que el matching semantico

UN ejemplo de SRI es el motor de busqueda de Google, o el buscador dentro de una biblioteca digital. Es importante notar que un SRI es diferente de un sistema de BDD por tres simples razones

- NO hay una respuesta exacta ni unica
- El objeto de trabajo es texto no estructurado
- La consulta no describe formalmente lo que se busca, sino que lo insinua

Componentes de un SRI

Los componentes de un SRI son los siguientes:

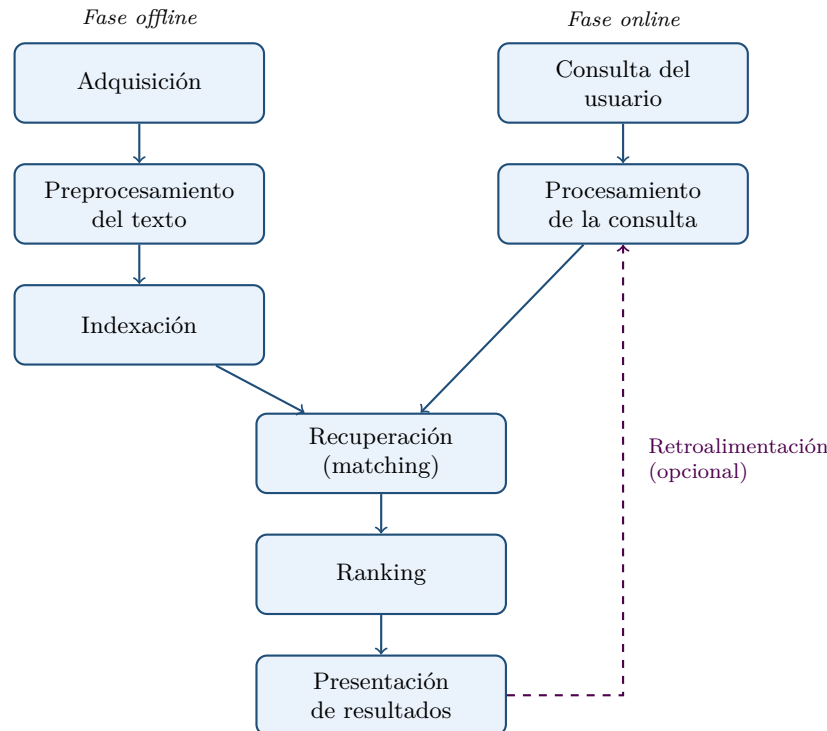
- Adquisición: Recopilación de documentos y construcción de la colección
- Preprocesamiento del texto: Limpieza, normalización, tokenización, stemming, eliminación de stopwords
- Indexación: Construcción de índices invertidos u otras estructuras de búsqueda. Este paso es de vital importancia porque lo que no se indexa, es como si no existiera.
- Procesamiento de la consulta: Transformación de la consulta del usuario en una forma que pueda ser comparada con los documentos
- Recuperación: Búsqueda de documentos candidatos en el índice
- Ranking: Ordenación de los documentos recuperados según su relevancia
- Presentación de resultados: Mostrar los documentos al usuario
- Retroalimentación: Ajuste de la consulta basado en la relevancia de los documentos mostrados

Cómo se conectan las fases?

Las fases no forman una sola fila, se dividen en dos ramas que corren por separado y convergen en el matching:

- **Rama de la colección (offline):** Adquisición, Preprocesamiento e Indexación construyen el índice invertido antes de que llegue ninguna consulta.
- **Rama de la consulta (online):** el usuario escribe su consulta y el sistema le aplica el mismo preprocesamiento que a los documentos, para que ambos lados queden en la misma forma y se puedan comparar.

Ambas ramas se encuentran en la Recuperación (matching), que usa el índice para hallar los documentos candidatos. De ahí en adelante el flujo es único: Ranking los ordena y Presentación de resultados se los muestra al usuario. La Retroalimentación, cuando existe, cierra el ciclo regresando al Procesamiento de la consulta para ajustarla con lo que el usuario marcó como relevante.



Preprocesamiento del texto

El preprocesamiento es la fase de limpieza que corre antes de indexar. Típicamente se quitan:

- Plurales
- Signos de puntuación
- Formato
- Stop words, o sea artículos, conjunciones y preposiciones

Después se hace **stemming** o **lematización**, que es básicamente dejar el texto hablando como cavernícola: se quitan diminutivos y terminaciones hasta quedarse con la raíz.

Idea clave

Hacer stemming es truncar las palabras estadísticamente. No se consulta ningún diccionario, se corta según reglas que en promedio funcionan, así que el pedazo que queda no siempre es una palabra real. *Corriendo, corrió y correremos* terminan los tres en *corr*.

La lematización hace el mismo trabajo pero por el camino caro: usa diccionario y análisis morfológico, así que devuelve el lema (*correr*, una palabra que sí existe) en lugar de un truncado.

Todo esto se hace por dos razones. La primera es que reduce el vocabulario, y el vocabulario es el que define el tamaño del índice. La segunda es que sin esto el matching falla por pura forma: la consulta dice *corrió* y el documento dice *corriendo*, que para una comparación de cadenas son dos cosas distintas.

Vector Space Model

Idea clave

Todo modelo de recuperación define dos cosas: como se guarda la información y como se recupera. El modelo de espacio vectorial contesta las dos con la misma herramienta, un vector.

Salton ideó representar los documentos como vectores, y desde entonces los documentos se representan como vectores en un espacio de N dimensiones, donde N es el número de términos de la colección.

Ojito...

Término no es lo mismo que palabra. Un término es lo que quedó después del preprocesamiento, o sea la raíz o el lema, así que varias palabras del texto original pueden colapsar en un solo término y contar como una sola dimensión.

La representación queda como una tabla donde cada renglón es un documento y cada columna un término. La celda guarda el peso de ese término en ese documento, que puede ser la frecuencia cruda o algo como TF-IDF.

	corr	perr	cas	com
d_1	3	0	4	0
d_2	0	2	0	4
d_3	1	1	2	0
q	1	1	0	0

La consulta se trata como cualquier otro documento: se preprocesa igual y se convierte en un vector del mismo espacio. Por eso q aparece como un renglón más de la tabla y no como algo aparte.

Idea clave

Aquí la relevancia se mide como similaridad. Un documento es relevante si su vector se parece al vector de la consulta.

La medida usual es la similitud coseno, que compara la dirección de los dos vectores e ignora su magnitud. Eso importa porque si no, un documento largo ganaría solo por ser largo:

$$\text{sim}(d, q) = \frac{\vec{d} \cdot \vec{q}}{\|\vec{d}\| \|\vec{q}\|} = \frac{\sum_{i=1}^N d_i q_i}{\sqrt{\sum_{i=1}^N d_i^2} \sqrt{\sum_{i=1}^N q_i^2}} \quad (2)$$

Con la tabla de arriba, q pide *corr* y *perr*. Sale primero d_3 con 0.58, luego d_1 con 0.42 y al final d_2 con 0.32. Gana d_3 porque cubre los dos términos de la consulta, aunque d_1 mencione *corr* tres veces: lo que d_1 tiene de más es *cas*, que a la consulta no le importa y solo le alarga el vector.

4. Continuación (Vector Space Model)

26 de agosto de 2026

Resumen rápido. Continuación del modelo de espacio vectorial. Primero de dónde viene, o sea las consultas booleanas que había antes. Luego el tema central de la sesión: cómo se llenan los vectores, con los tres esquemas de ponderación y el principio de que la importancia de un término decrece conforme aparece en más documentos. Cerramos con los pros y contras del enfoque y con un repaso de los otros modelos de recuperación.

De dónde viene: las consultas booleanas

Antes del modelo de espacio vectorial estaban las consultas booleanas, que fueron la primera forma de recuperar información. La consulta se armaba con operadores booleanos: quiero esto **y** esto, esto **o** lo otro, esto pero **no** aquello.

El vector space model rompe con eso: ya no representa la consulta ni los documentos como expresiones lógicas, sino como vectores. Al inicio fue con álgebra vectorial simple, y después se extendió a cosas más complejas.

Idea clave

El cambio de fondo no es la notación sino el resultado. Una consulta booleana parte la colección en dos, los documentos que cumplen y los que no. Un vector permite ordenar por qué tanto se parecen, y eso es lo que da el ranking.

Cómo se llenan los vectores

La forma en que se llenan los vectores sirve para describir la importancia de un término en el documento. Puede ser el número de veces que aparece la palabra, o alguna variante de eso.

El principio que rige todo el pesado es este: la importancia general de un término decrece proporcionalmente a su ocurrencia en la colección entera. Hay atributos, palabras y términos que son tan frecuentes que no sirven para discriminar.

Ojito...

Esos términos tan frecuentes no se eliminan en el procesamiento. No hace falta: el esquema de pesado se encarga de dejarlos en un peso cercano a cero, así que se anulan solos.

Hay muchos métodos para llenar la matriz, pero todos se basan en lo mismo, la ponderación por número de veces y la importancia que decrece.

Binary weights

El peso vale 1 si el término aparece en el documento y 0 si no. Solo registra presencia, así que un término que sale una vez pesa lo mismo que uno que sale cincuenta.

Term frequency

Se usa la frecuencia, pero normalizada. Con $f_{i,j}$ el número de veces que el término i aparece en el documento j ,

$$tf_{i,j} = \frac{f_{i,j}}{\max_k f_{k,j}},$$

donde el denominador es la frecuencia del término más frecuente de ese documento. Se normaliza porque si no, un documento largo tendría pesos más grandes solo por ser largo.

TF-IDF

Es el esquema estándar, y combina las dos mitades del principio: qué tan seguido sale el término en *este* documento, contra en cuántos documentos de la colección sale. Con N el total de documentos y n_i cuántos contienen al término i ,

$$w_{i,j} = \text{tf}_{i,j} \times \text{idf}_i, \quad \text{idf}_i = \log \frac{N}{n_i}.$$

Idea clave

El idf es la formalización de que la importancia decrece. Un término que aparece en todos los documentos tiene $n_i = N$, así que $\log(N/n_i) = \log 1 = 0$ y su peso se anula sin que nadie lo haya quitado a mano.

Okapi BM25

Es un refinamiento de TF-IDF, y viene del sistema Okapi. Arregla dos cosas que TF-IDF hace mal.

La primera es que en TF-IDF la frecuencia entra de forma lineal, así que un término que aparece cien veces pesa cien veces más que uno que aparece una. Eso no corresponde a cómo funciona la evidencia: que una palabra pase de aparecer una vez a dos dice muchísimo sobre el tema del documento, pero que pase de cincuenta a cincuenta y uno ya no dice casi nada.

La segunda es que la normalización por longitud de TF-IDF es fija. BM25 la vuelve ajustable, porque no siempre conviene normalizar igual: un documento puede ser largo por tratar más temas, o por ser verboso sobre el mismo.

Con $|d|$ la longitud del documento en términos y avgdl la longitud promedio de la colección,

$$\text{BM25}(d, q) = \sum_{i \in q} \text{idf}_i \cdot \frac{f_{i,d} (k_1 + 1)}{f_{i,d} + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}} \right)}.$$

Los dos parámetros son los que se ajustan:

- k_1 controla qué tan rápido satura la frecuencia. Con $k_1 = 0$ el modelo se vuelve binario, solo importa que el término esté; conforme k_1 crece se parece más a la frecuencia cruda. Lo típico es entre 1.2 y 2.0.
- b controla la normalización por longitud. Con $b = 0$ no normaliza nada y con $b = 1$ normaliza por completo. Lo típico es 0.75.

Idea clave

La saturación es la idea central. Cuando $f_{i,d}$ crece sin límite, la fracción tiende a $k_1 + 1$ y ahí se queda, así que la contribución de un término está acotada por más veces que aparezca. TF-IDF no tiene tope.

A pesar de ser de los noventa, BM25 sigue siendo el baseline fuerte contra el que se comparan los métodos neuronales de recuperación, y es lo que usan por defecto motores como Lucene y Elasticsearch.

Qué no captura el modelo

Esta representación no considera la semántica de los documentos. El vector solo sabe qué términos hay y con qué peso, no qué significan.

A favor:

- Es fácil de explicar
- Está matemáticamente bien estructurado
- Permitió el *approximate query matching*, o sea recuperar documentos que no calzan exacto con la consulta

En contra:

- No entiende la sinonimia: *carro* y *auto* son dos dimensiones distintas y ortogonales
- No entiende la semántica
- Las consultas estructuradas son difíciles de modelar
- La normalización aumenta el costo computacional
- Hay que pesar los términos, y de ahí salen los esquemas de arriba

Otros modelos de recuperación

- **Modelos booleanos.** Consulta con operadores lógicos, sin ranking: un documento cumple o no cumple.
- **Probabilistic indexing.** Pesa los términos del índice por su probabilidad de ser relevantes.
- **Vector space model.** El que estamos viendo.
- **Probabilistic retrieval.** Ordena los documentos por la probabilidad de que sean relevantes para la consulta.
- **Fuzzy set models.** La pertenencia al conjunto de documentos relevantes es gradual y no de todo o nada.
- **Inference networks.** Modelan la recuperación como inferencia sobre una red bayesiana.
- **Language models.** Cada documento define un modelo de lenguaje, y se ordena por la probabilidad de que ese modelo genere la consulta.
- **Word embeddings y enfoques neuronales.** Representan términos y documentos en un espacio denso aprendido, que sí captura sinonimia y semántica. Es la respuesta directa a las dos contras de arriba.

5. Modelos de lenguaje para IR

31 de agosto de 2026

Resumen rápido. En esta sesión vimos como se utilizan los modelos de lenguaje para la Info Retrieval y como se evalúa el desempeño de un sistema de IR, nos quedamos deduciendo una fórmula de evaluación de desempeño... la "Arturiana" y se quedó de tarea como desarrollarla completamente.

Modelos de lenguaje estadísticos

Un modelo de lenguaje estadístico asigna una probabilidad a una secuencia de m palabras por medio de una distribución de probabilidad. La secuencia se descompone con la regla de la cadena:

$$P(t_1, t_2, \dots, t_m) = P(t_1) P(t_2 | t_1) P(t_3 | t_1 t_2) \cdots = \prod_{i=1}^m P(t_i | t_1, \dots, t_{i-1}).$$

Cada término se condiciona a todo lo que vino antes, que es justo lo que vuelve el cálculo impráctico: para contextos largos casi ninguna secuencia se repite en el corpus. Lo que dice es,

$$P(t_1, t_2, \dots, t_m)$$

la probabilidad de que aparezca la secuencia de palabras en ese orden $t_1, t_2, \dots, t_m$.

Es igual a

$$P(t_1) P(t_2 | t_1) P(t_3 | t_1 t_2) \cdots P(t_m | t_1 \dots t_{m-1})$$

La probabilidad de que salga la primera palabra, por la probabilidad de que salga la segunda dado la primera y así... Pero desarrollando los términos con la definición de probabilidad condicional, resulta

$$P(t_1, t_2, \dots, t_m) = P(t_1) \frac{P(t_1, t_2)}{P(t_1)} \frac{P(t_1, t_2, t_3)}{P(t_1, t_2)} \cdots \frac{P(t_1, \dots, t_m)}{P(t_1, \dots, t_{m-1})} = \prod_{i=1}^m P(t_i | t_1, \dots, t_{i-1}).$$

Secuencia de unigramas

La forma más simple de que un modelo de lenguaje deseché el contexto probabilístico es tratar cada término de manera independiente. Esto sirve por si no hay mucho contexto, y se llama secuencia de unigramas:

$$P(t_1, t_2, \dots, t_m) = \prod_{i=1}^m P(t_i).$$

De dónde se sacan esas probabilidades?

Se estiman contando sobre el texto. Para el modelo M_d construido sobre el documento d , la estimación por máxima verosimilitud es la frecuencia relativa del término:

$$P(t | M_d) = \frac{\text{tf}_{t,d}}{|d|},$$

o sea las veces que t aparece en d entre el total de términos de d .

Antes de entrar a cómo se usan en recuperación de información, vale notar que un modelo de lenguaje se usa en el día a día: en los autocompletados, traducción de texto, corrección ortográfica y reconocimiento de habla.

Cómo se usan los LM en IR

La idea principal es que los documentos pueden ser rankeados por su similitud de generar la consulta.

A cada documento lo voy a ver como si fuese un mundo, algo independiente, y sobre ese documento se calcula o se construye un modelo de lenguaje. Cuando llega una consulta se hace la pregunta de cuál es la probabilidad de generar la secuencia de la consulta a partir de cada modelo de lenguaje:

$$P(Q | M_{d_i}) = \prod_{t \in Q} P(t | M_{d_i}).$$

El objetivo es rankear los documentos por $P(d | Q)$, donde la probabilidad de un documento se interpreta como qué tan similar o relevante es a la consulta. Aplicando Bayes:

$$P(d_i | Q) = \frac{P(Q | d_i) P(d_i)}{P(Q)} \propto P(Q | d_i) P(d_i) \approx P(Q | M_{d_i}).$$

Ojito...

$P(Q)$ se puede tirar porque es la misma para todos los documentos: divide a todos por igual y no cambia el orden. Por eso la igualdad se vuelve proporcionalidad. Y si además se toma $P(d_i)$ uniforme, rankear por $P(d | Q)$ es lo mismo que rankear por $P(Q | M_{d_i})$.

$P(d)$ es la confianza, algo así como la probabilidad de que ese documento sea recuperado antes de ver la consulta. Ahí es donde entra información que no depende de la consulta, por ejemplo qué tan citado o qué tan reciente es el documento.

Qué pasa cuando veo un término nuevo

Si un término de la consulta no aparece en el documento, su tf es cero, y como el producto multiplica todos los términos, la probabilidad entera se va a cero. Un solo término no visto descarta el documento aunque los demás calcen perfecto.

Para solucionar esto se asume que los términos no vistos se vieron una vez:

$$P(t | M_d) = \frac{\text{tf}_{t,d} + 1}{|d| + |V|},$$

con $|V|$ el tamaño del vocabulario.

Pero una forma más robusta es agregar un modelo de lenguaje para toda la colección completa, o sea hacer una mezcla entre el documento y la colección. Con M_C el modelo de la colección y $\lambda \in (0, 1)$ el peso que se le da al documento, la fórmula de ranking queda:

$$P(d | Q) \propto P(d) \prod_{t \in Q} [\lambda P(t | M_d) + (1 - \lambda) P(t | M_C)].$$

Idea clave

La mezcla nunca deja un factor en cero, porque si el término no está en el documento el segundo sumando lo rescata con lo que sabe la colección. Y de paso pesa mejor: un término raro en la colección tiene $P(t | M_C)$ chica, así que encontrarlo en el documento cuenta más. Es el mismo efecto del idf, pero saliendo de la probabilidad en lugar de estar puesto a mano.

Evaluación de IR

Lo primero que se puede medir es la **eficiencia**: qué tanto tiempo se tarda el modelo. Otra cosa que se puede medir es cuántos relevantes trajo y cuántos relevantes olvidó.

Se puede decir que el IR es subjetivo, porque la relevancia de algo es subjetiva, pero se hace el etiquetado con muchas personas para poder llegar a un consenso.

Cómo funciona el sistema?

La pregunta se contesta en tres niveles:

- **Procesamiento.** Qué tanto tiempo se tardó y el espacio que usó.
- **Búsqueda.** Qué tan buenos son los resultados.
- **Sistema.** La satisfacción del usuario.

Nos enfocaremos en evaluar la **efectividad** de recuperación, o sea el segundo nivel.

El problema de la relevancia

El problema de la efectividad es que está relacionada con la relevancia de los ítems recuperados, y la relevancia es escurridiza:

- No es binaria, es un valor continuo
- Incluso si fuese binaria, podría ser muy difícil de juzgar

Además depende de varias cosas a la vez:

- La **necesidad** de información del usuario
- **Situacional**, relacionado con esa necesidad y con el contexto en el que busca
- **Cognitiva**, depende de la percepción humana y del comportamiento
- **Dinámica**, cambia en el tiempo

Ojito...

La relevancia es subjetiva y depende de muchos valores. Por eso no se evalúa preguntándole a una sola persona, sino juntando juicios de varias hasta que haya consenso.

Paradigma de Cranfield

Es la forma estándar de evaluar, y consiste en armar una colección de prueba (*test collection*). Para que sirva hace falta:

- Muchos documentos, entre más mejor
- Muchas consultas
- *Relevance judgments* para todas las consultas

Se pone a una persona a hacer la recuperación sobre esa colección y sus resultados se van evaluando contra los juicios de relevancia. Como la colección, las consultas y los juicios quedan fijos, dos sistemas distintos se pueden comparar de forma justa.

Colecciones de prueba estándar

- TREC
- CLEF
- NTCIR

Tipos de medidas

Hay dos tipos:

- **Basadas en conjuntos.** Tenía que recuperar estos, pero me dio estos. Solo miran qué documentos salieron, no en qué orden.
- **Basadas en ranking.**Cuál es más importante que cuál. Aquí sí importa la posición, porque no es lo mismo que el relevante salga primero a que salga en el lugar cincuenta.